Solution Design Document

Project: CII Threat Scenario Generator **Document ID:** SDD-CTSG-001 **Version:** 3.0 **Status:** For Client Review **Date:** 2026-05-18 **Classification:** Confidential — Client Restricted

---

Table of Contents

1. Document Control

1.1 Version History

Version	Date	Author	Reviewer	Summary of Changes
0.1	2026-05-15	Solution Architect	—	Initial draft
1.0	2026-05-16	Solution Architect	Tech Lead	First client-review draft
2.0	2026-05-17	Solution Architect	Tech Lead	Restructured to standard SDD section order
3.0	2026-05-18	Solution Architect	Tech Lead, Security Architect	Added document control, formal requirements, use cases with sequence diagrams, interface contracts, ER diagram, deployment diagram, risk register, open issues, appendices

1.2 Approvers

Role	Name	Signature	Date
Client — Programme Sponsor	TBD		
Client — Security Architect	TBD		
Client — Solution Architect	TBD		
Vendor — Solution Architect	TBD		
Vendor — Delivery Manager	TBD		

1.3 Reviewers

Role	Name	Review Cycle
Client — Compliance Officer	TBD	v3.0
Client — Infrastructure Lead	TBD	v3.0
Client — Integration Architect	TBD	v3.0
Vendor — Tech Lead	TBD	v3.0
Vendor — Security Architect	TBD	v3.0

1.4 Distribution List

Recipient	Organisation	Role
Programme Sponsor	Client	Decision authority
Security Architecture team	Client	Design review
Integration Architecture team	Client	Interface review
Compliance Office	Client	Audit & regulatory review

Recipient	Organisation	Role
Delivery Team	Vendor	Implementation

1.5 Related Documents

Ref	ID	Document	Owner
R1	BRD-CTSG-001	Business Requirements Document	Client
R2	FRS-CTSG-001	Functional Requirements Specification	Client
R3	NFR-CTSG-001	Non-Functional Requirements Specification	Client
R4	DDS-CTSG-001	Database Design Specification (`docs/SEED_DATA_DATABASE_SCHEMA.md`)	Vendor
R5	OPS-CTSG-001	Operations Runbook Set (`docs/RUNBOOKS/`)	Vendor
R6	SOL-CTSG-001	Internal Solution Document (`SOLUTION_DOCUMENT.md`)	Vendor

2. Introduction

2.1 Purpose

This Solution Design Document (SDD) specifies the design of the CII Threat Scenario Generator. It is the authoritative technical reference for the design of the application, data, interface, security, and deployment layers. It defines *how* the requirements stated in R1–R3 are realised.

2.2 Scope

In Scope	Out of Scope
Cascading metadata APIs serving the six UI pickers	Live threat-intelligence ingestion
Synchronous and asynchronous scenario generation	Active penetration testing or exploitation
Deterministic twelve-step pipeline with one constrained LLM call	Automated remediation execution
Deterministic Attack Tree construction	End-user authentication (delegated to client)
Risk scoring, control coverage, evidence aggregation	Threat library curation tooling
Per-tenant rate limiting and audit isolation	Client SIEM / APM dashboard implementation
Pluggable DB (MSSQL / PostgreSQL / SQLite), LLM (OpenAI / Azure OpenAI / Mock), cache (in-process / Redis)	Production-grade load balancing topology

2.3 Intended Audience

Audience	Sections of Interest
Programme Sponsor	§2, §3, §13 (Risks)
Security Architect	§3.3, §6, §9, §10, §13
Integration Architect	§4, §6, §8, §11
Solution Architect	All
Compliance Officer	§3, §7, §9, §10, §13
Infrastructure Lead	§5, §10, §11, §12
Vendor Implementation Team	All

2.4 Definitions, Acronyms, Abbreviations

Term	Definition
CII	Critical Information Infrastructure
SDD	Solution Design Document
BRD	Business Requirements Document
FRS	Functional Requirements Specification
NFR	Non-Functional Requirement

Term	Definition
DDS	Database Design Specification
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
LLM	Large Language Model
DI	Dependency Injection
RTM	Requirements Traceability Matrix
SLA	Service Level Agreement
RPO / RTO	Recovery Point Objective / Recovery Time Objective
TLS	Transport Layer Security
IdP	Identity Provider
APM	Application Performance Monitoring
SIEM	Security Information and Event Management
Master Data	Curated reference catalogue (assets, threats, controls, rules)
Scenario ID	Deterministic content-derived ID: `GEN-{SECTOR}-{ASSET}-{CATEGORY}-{HASH}`
Task ID	UUID assigned to an asynchronous submission
Attack Tree	Five-lane node/edge graph (Actor → Action → Vector → Condition → Impact)

2.5 References

External standards and frameworks referenced in this design:

Ref	Title
IEEE 1016-2009	Standard for Information Technology — Systems Design — Software Design Descriptions
ISO/IEC 27001:2022	Information Security Management Systems — Requirements
OWASP API Security Top 10 (2023)	API security risk categories
W3C Trace Context	Distributed tracing propagation specification

3. Requirements Summary and Traceability

3.1 Functional Requirements

ID	Requirement	Priority
FR-001	The system shall accept exactly six fields from the user via the UI: Asset, Asset Type, Threat Category, Threat Type, Threat, Threat Actor	Must
FR-002	The system shall enrich the user-supplied fields with Sector, Sub-sector, Service, asset operational profile, recommended controls, evidence, applicability rules, and scoring rules from the master database	Must
FR-003	The system shall expose cascading metadata endpoints to power the six UI pickers	Must
FR-004	The system shall expose an asynchronous endpoint for single-scenario generation, returning a Task ID	Must
FR-005	The system shall expose a synchronous endpoint for batch generation of up to 50 scenarios per request	Must
FR-006	The system shall produce a structured Threat Scenario narrative for each accepted submission	Must
FR-007	The system shall produce a deterministic Attack Tree for each accepted submission	Must
FR-008	The system shall compute Likelihood, Impact, Priority Score, and Priority Rating per scenario	Must
FR-009	The system shall assign a stable, content-derived Scenario ID (`GEN-…`) such that identical inputs yield identical IDs	Must
FR-010	The system shall reject any threat whose technology requirements are not met by the asset (technology gate)	Must
FR-011	The system shall persist every submission, response, and lifecycle transition to an audit table	Must
FR-012	The system shall return paginated audit history filtered by tenant	Must
FR-013	The system shall expose liveness and readiness health probes	Must
FR-014	The system shall flag scenarios for human review where the Priority Rating is High/Critical, Applicability is Conditional, Evidence is Pending, or the Asset is High-criticality	Must
FR-015	The system shall reject LLM output containing exploit instructions, payloads, bypass techniques, or credential-theft instructions	Must

3.2 Non-Functional Requirements

ID	Requirement	Target
NFR-001	Availability	99.5% monthly (excluding planned maintenance)
NFR-002	Scenario generation latency (P95)	≤ 30 s (dominated by LLM call)

ID	Requirement	Target
NFR-003	Metadata read latency (P95)	≤ 200 ms
NFR-004	Per-tenant concurrent task limit	Configurable; default 20
NFR-005	Read-endpoint sliding-window limit	Configurable; default 120 requests / 60 s per (tenant, route)
NFR-006	Maximum request body size	Configurable; default 256 KB; HTTP 413 on excess
NFR-007	Database portability	MSSQL, PostgreSQL, SQLite
NFR-008	LLM portability	OpenAI, Azure OpenAI, Mock
NFR-009	Multi-tenancy	Isolated audit and rate limits per tenant
NFR-010	Auditability	100% of submissions persisted with request, response, status, timestamps
NFR-011	Observability	Structured JSON logs; X-Request-ID / W3C traceparent propagation; Sentry-compatible APM seam
NFR-012	Container security	Non-root user, multi-stage image, embedded healthcheck
NFR-013	LLM resilience	Per-call timeout + circuit breaker (closed / open / half-open)
NFR-014	Cache portability	In-process default; Redis-shared opt-in; namespaced by schema version

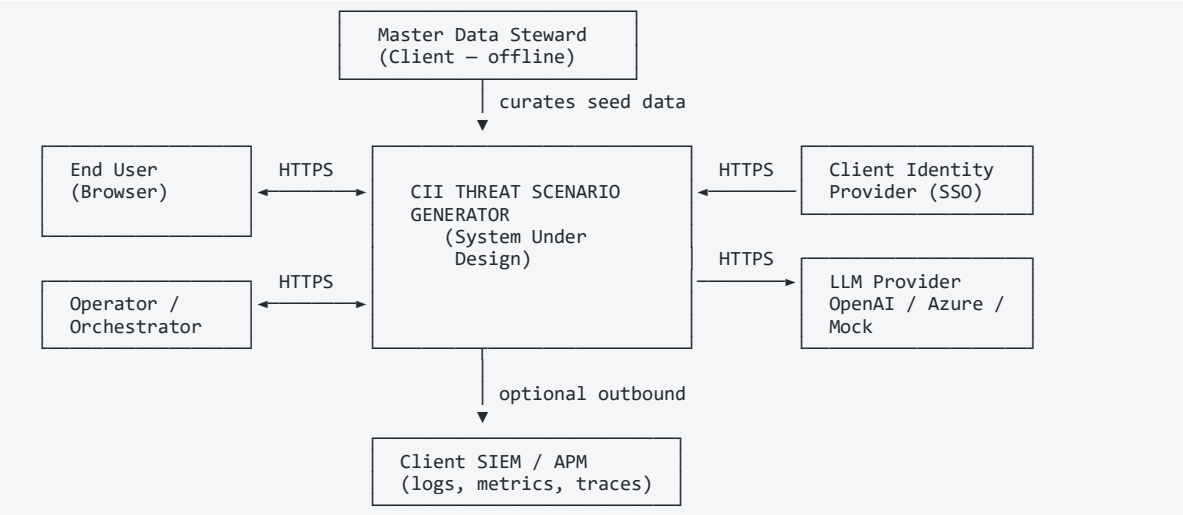
3.3 Requirements Traceability Matrix

Requirement	Realised By (Design Element)	SDD Section
FR-001	Input Model — 6 fields	§6.2
FR-002	Pipeline Step 0 — DB Enrichment	§6.5
FR-003	Cascading Metadata Endpoints	§8.2.1–§8.2.7
FR-004	`POST /generate-scenario` + Celery worker	§8.2.8
FR-005	`POST /generate-scenario/batch`	§8.2.10
FR-006	Pipeline Step 11 + ScenarioOutput model	§7.5
FR-007	Pipeline Step 12 + AttackTreeOutput model	§7.6
FR-008	Pipeline Step 7 — Ranking Engine	§6.5
FR-009	Pipeline Step 6 — Scenario ID Builder	§7.7
FR-010	Pipeline Step 5 — Applicability Engine	§6.5
FR-011	Audit data model `generated_scenarios`	§7.4
FR-012	`GET /scenarios/history`	§8.2.11
FR-013	`GET /health`, `GET /health/ready`	§8.2.12, §8.2.13

Requirement	Realised By (Design Element)	SDD Section
FR-014	Post-Step Safety Validator	§6.5
FR-015	Post-Step Safety Validator — unsafe-term scan	§6.5, §9.4
NFR-001	Stateless API + horizontally scalable workers	§5, §11
NFR-002	Single LLM call + circuit breaker	§6.5, §10.1
NFR-003	In-memory master-data cache	§5.3 C7
NFR-004	Per-tenant counter (Redis)	§10.2
NFR-005	Sliding-window read limiter (Redis)	§10.2
NFR-006	Request size middleware	§8.5
NFR-007	SQLAlchemy Core + dialect-portable DDL	§7.3
NFR-008	Provider-pluggable `LLMClient`	§5.3 C12
NFR-009	Tenant-scoped middleware + audit table indexes	§9.1, §7.4
NFR-010	Audit table writes per task	§7.4
NFR-011	JSON logger + request-context middleware + Sentry seam	§10.4
NFR-012	Hardened Dockerfile	§11.3
NFR-013	`llm_circuit_breaker.py`	§5.3 C12
NFR-014	`CacheBackend` protocol	§5.3 C7

4. Solution Overview

4.1 Solution Context Diagram



4.2 Actors

Actor	Type	Description
End User	Human	Security analyst or risk officer who generates threat scenarios
Master Data Steward	Human	Curates assets, threats, controls, and rules in master data
Operator / Orchestrator	System	Container orchestrator (e.g., Kubernetes) probing health endpoints
Client Identity Provider	System	Authenticates the end user; injects verified `tenant_id`
LLM Provider	System	Executes the single language-generation call per scenario
Client SIEM / APM	System	Optional consumer of structured logs and traces

4.3 Use Case Inventory

UC	Name	Primary Actor	Section
UC-01	Cascade through master-data pickers	End User	§6.1
UC-02	Generate single threat scenario (async)	End User	§6.2
UC-03	Generate batch of threat scenarios (sync)	End User	§6.3
UC-04	Retrieve generated scenario by Task ID	End User	§6.4
UC-05	Browse scenario history	End User	§6.5
UC-06	Probe service health	Operator	§6.6

4.4 High-Level Design Decisions

ID	Decision	Rationale
HLD-01	Six user-supplied fields; everything else DB-derived	Eliminates free-text input, prevents hallucination, simplifies audit
HLD-02	Deterministic pipeline with single constrained LLM call	Bounds the LLM's authority; risk scoring and the attack tree are non-LLM
HLD-03	Asynchronous task model via Celery for single submissions	Decouples LLM latency from the API request thread
HLD-04	Synchronous batch endpoint for bulk submissions	Enables integration tooling and pre-warmed data flows
HLD-05	Port-Adapter (Dependency Injection) across DB, cache, LLM, embeddings	Backends are swappable without pipeline changes
HLD-06	In-memory master-data cache with optional Redis-shared backend	Sub-millisecond reads, scalable across worker pools
HLD-07	Stable Scenario ID derived from input hash	Same inputs produce the same ID; enables before/after comparison
HLD-08	Multi-tenant isolation via tenant-aware middleware	Single deployment serves multiple client tenants safely

5. Application Architecture

5.1 Architectural Style

A layered, service-oriented architecture with strict separation of concerns and dependency injection between layers. The Domain (pipeline) layer has no direct knowledge of database, cache, queue, or LLM implementations.

Layer	Responsibility	Dependency Direction
Presentation	UI rendering and orchestration	Calls API layer over HTTPS
API	HTTP endpoints, request validation, middleware	Calls Domain layer via DI
Domain (Pipeline)	Deterministic engines, LLM call, validators	Calls Adapter layer via Ports (Protocols)
Adapter	DB, cache, LLM, embedding adapters	Calls Infrastructure
Infrastructure	Databases, Redis, embedding model, LLM provider	—

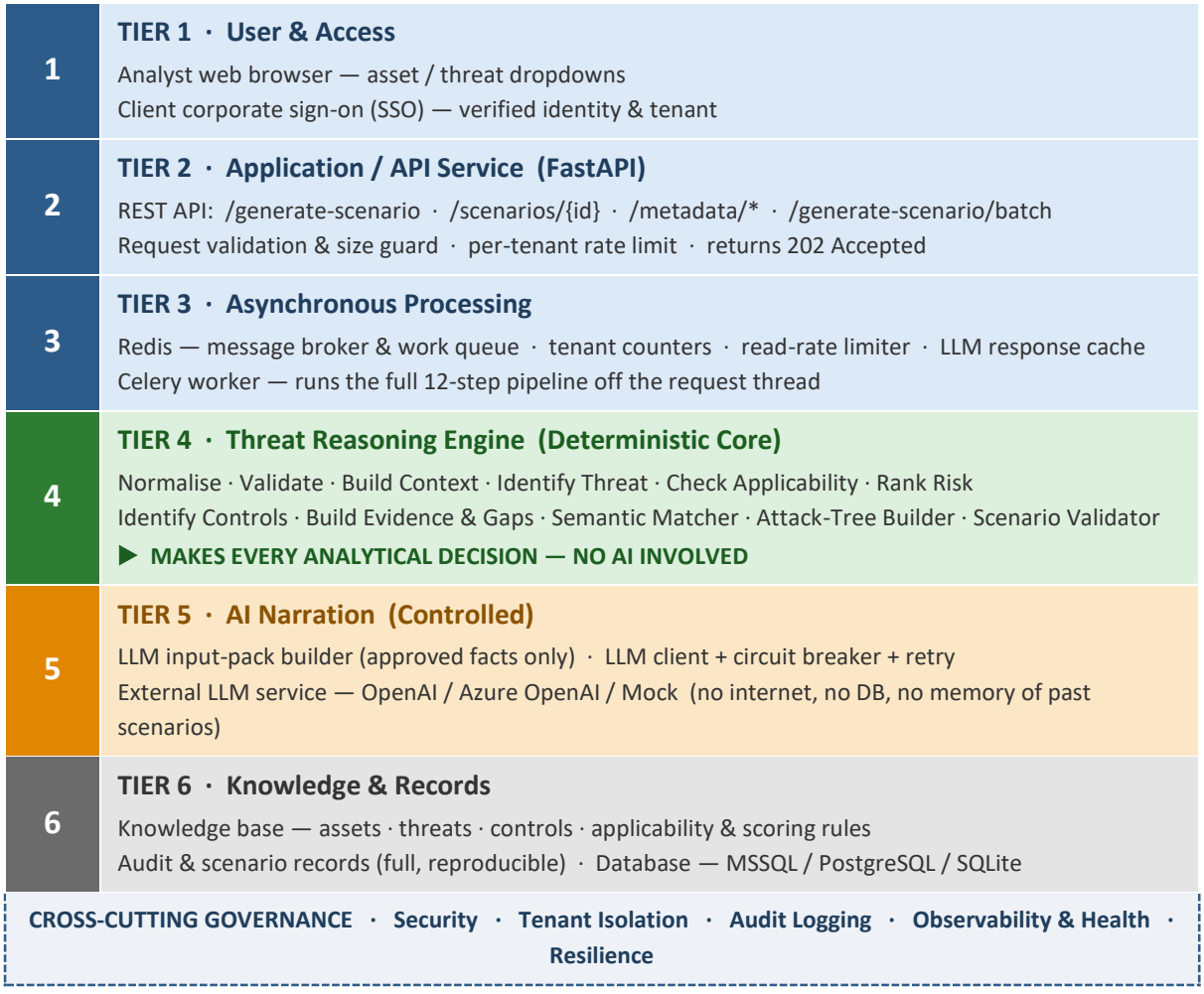


Figure 5-1 — System Architecture. Six runtime tiers; the green Deterministic Core makes every analytical decision and the amber AI tier writes prose once per scenario. Native Word table — click any cell to edit text, recolour via Table Design › Shading, or add rows.

5.2 Logical Architecture Diagram

U	**End User · Web Browser** Security analyst working in a standard browser — nothing to install on the client device.
G	**Client Edge / Identity Gateway (client-owned)** SSO · mTLS · API Gateway — authenticates the user and injects tenant_id on every request.
1	**Presentation Layer — Web UI** Cascading pickers · context panel · scenario & attack-tree rendering · evidence checklist. ↓ *depends on → the API Layer, called over HTTPS*
2	**API Layer — FastAPI** HTTP endpoints (metadata · scenario · batch · history) and cross-cutting middleware: tenant routing · X-Request-ID · X-API-Key · size guard · rate limiters · CORS. ↓ *depends on → the Domain Layer, wired by dependency injection*
3	**Domain / Pipeline Layer (the core)** 12 deterministic engines + one constrained LLM call + the safety validator. ▶ **Pure functions over typed models — no knowledge of DB, cache, queue or LLM internals.** ↓ *depends on → the Adapter Layer, reached only through Ports (Python Protocols)*
4	**Adapter Layer** Concrete implementations behind the Ports: DB repositories · cache backend · LLM client + circuit breaker · embedding client · audit logger · tenant counter. ↓ *depends on → the Infrastructure Layer*
5	**Infrastructure Layer** Master DB & Audit DB (MSSQL / PostgreSQL / SQLite) · Redis (broker · cache · counters) · local embedding model · LLM provider (OpenAI / Azure OpenAI / Mock).

Figure 5-2 — Logical Architecture. Five layers with strictly one-directional dependencies (top to bottom); the Domain Layer holds all logic yet knows nothing of databases, cache, queue or LLM — it reaches them only through Ports. Native Word table — fully editable.

5.3 Component Specifications

ID	Component	Layer	Responsibility	Key Interfaces
C1	Web UI	Presentation	Cascading pickers, context panel, scenario/tree rendering	HTTPS → API
C2	Metadata Endpoints	API	Filtered picker options	`GET /metadata/*`
C3	Scenario Submit Endpoint	API	Six-field validation, audit row creation, task enqueue	`POST /generate-scenario`
C4	Poll & History Endpoints	API	Read persisted responses; paginate history	`GET /scenarios/{task_id}`, `GET /scenarios/history`
C5	Batch Endpoint	API	Synchronous bulk pipeline execution	`POST /generate-scenario/batch`
C6	API Middleware	API	Tenant routing, X-Request-ID, X-API-Key, size guard, read limiter, concurrent limiter, CORS	Cross-cutting

ID	Component	Layer	Responsibility	Key Interfaces
C7	Master-Data Cache	Cache	Cached master data	`CacheBackend` protocol (`InProcessCache` / `RedisCache`)
C8	Celery Worker Pool	Queue	Async task execution	Celery task contract
C9	Pipeline Engines	Domain	Steps 0–12	Internal Python interfaces
C10	Safety Validator	Domain	Output validation, unsafe-term scan, human-review flags	Internal
C11	Attack Tree Builder	Domain	Convert `ScenarioOutput` → `AttackTreeOutput`	Internal
C12	LLM Client + Circuit Breaker	Adapter	Single LLM call with timeout and breaker	`LLMClient` protocol
C13	Embedding Client	Adapter	Cosine-similarity matcher	`EmbeddingClient` protocol
C14	DB Adapters	Adapter	Typed Protocol over master tables	`SeedRepositories` bundle
C15	Audit Logger	Adapter	Read/write to `generated_scenarios`	`AuditLogger` protocol
C16	Tenant Counter	Adapter	Atomic Redis counter for rate limit	`TenantCounter` protocol
C17	Master DB	Infrastructure	Source of master data	SQL via SQLAlchemy Core
C18	Audit DB	Infrastructure	Audit row store	SQL via SQLAlchemy Core
C19	Redis	Infrastructure	Broker + cache + counters	Redis protocol
C20	LLM Provider	External	Generation service	HTTPS
C21	Client Identity Gateway	External	Authentication	HTTPS

5.4 Technology Stack

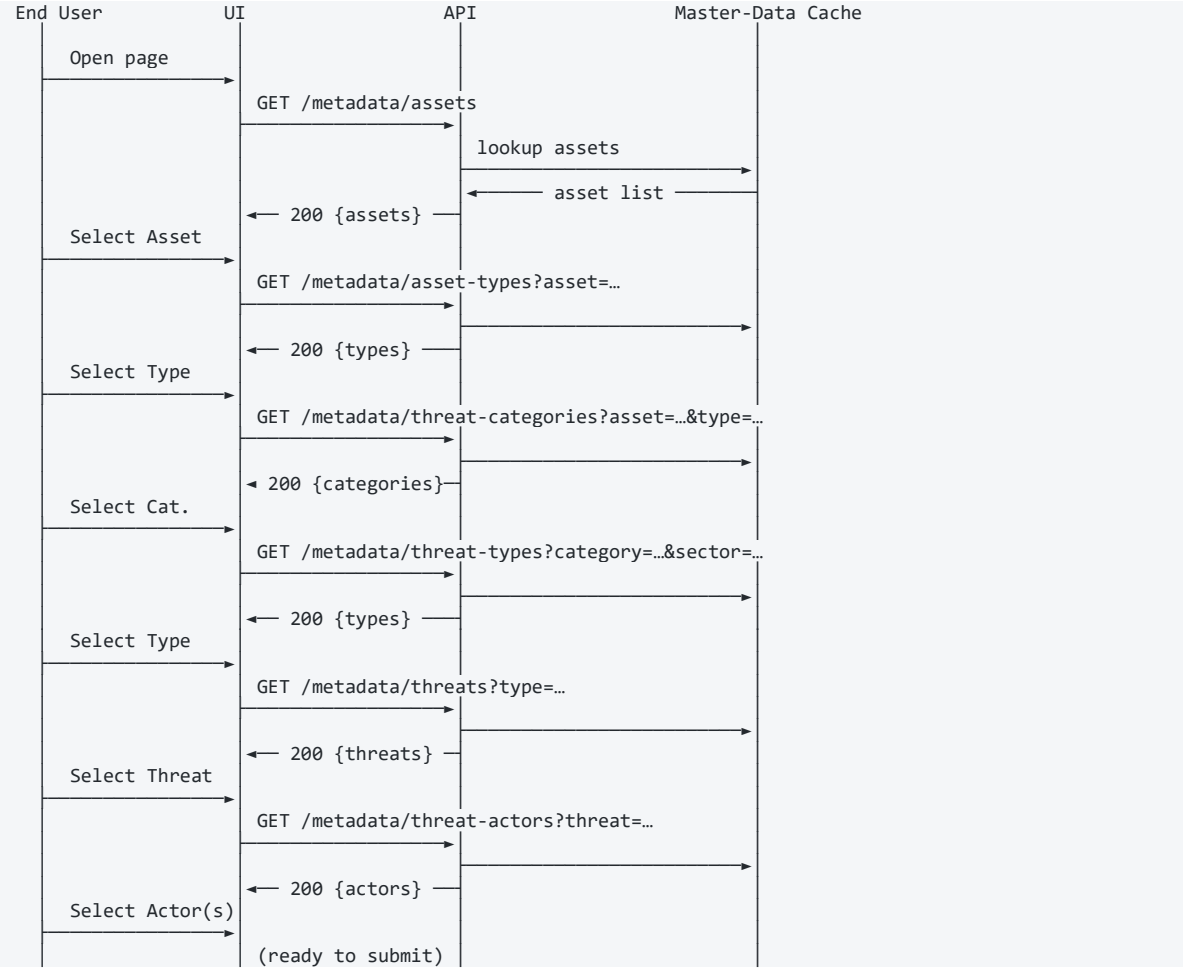
Layer	Technology	Version
Language	Python	3.12
API framework	FastAPI on Uvicorn	Latest stable
Task queue	Celery	5.x
ORM	SQLAlchemy Core	2.x
Database (selectable)	MSSQL / PostgreSQL / SQLite	Vendor-defined
Cache / Broker	Redis	7.x
Embedding	`sentence-transformers` running `multilingual-e5-large`	Local CPU
LLM Provider	OpenAI Responses / Azure OpenAI Chat Completions / Mock	Provider-defined
Container	Multi-stage Dockerfile on `python:3.12-slim`	—
Observability	Structured JSON logs + optional Sentry SDK	—

6. Behavioural Design

6.1 UC-01 — Cascade through master-data pickers

Primary Actor: End User **Trigger:** User opens the Scenario Generator. **Preconditions:** User is authenticated; tenant_id is injected by the gateway. **Postconditions:** All six fields are selected; payload is ready for submission.

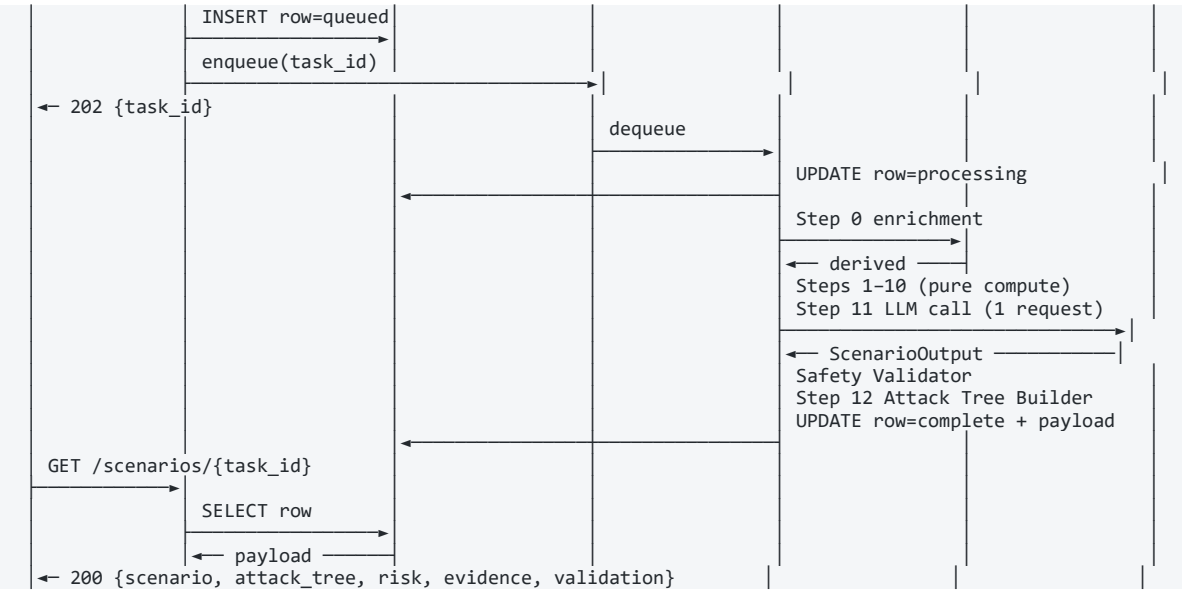
Main Flow Sequence Diagram:



Alternate flow: Any picker may return an empty list if no records match the cascading filter; UI displays "no options" and blocks progression.

6.2 UC-02 — Generate single threat scenario (async)

Primary Actor: End User **Trigger:** User clicks **Generate** after completing UC-01. **Preconditions:** Six valid fields selected; tenant is below concurrent task limit. **Postconditions:** Audit row persisted with status `complete` (or `failed`); response payload populated.

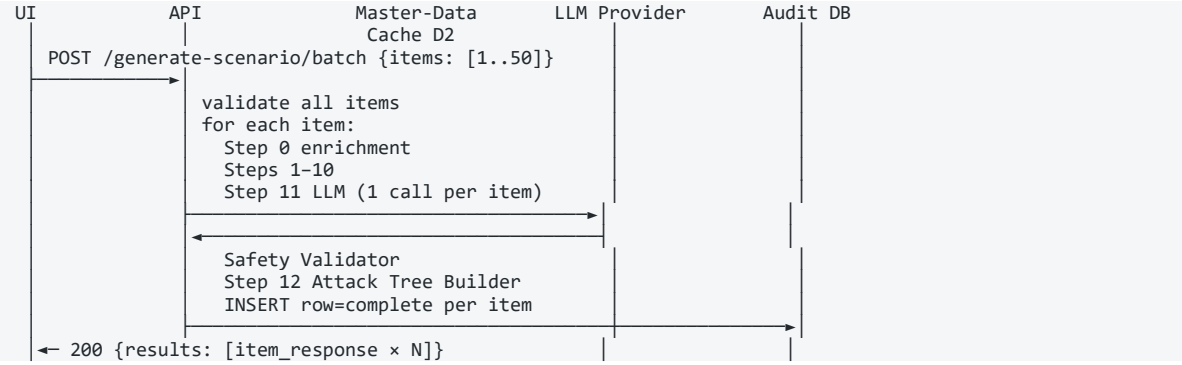


Exception flows:

Condition	Outcome
Validator (Step 2) detects unknown DB value	Pipeline raises `ScenarioValidationError`; row → `failed`; HTTP 422 returned on poll
Applicability Engine (Step 5) returns `Rejected`	Pipeline stops at Step 6; LLM not called; row → `complete` with `status="Rejected"`; HTTP 200
LLM provider unavailable or times out	Circuit breaker opens; row → `failed`; HTTP 502/503 returned on poll
Safety validator detects unsafe content	Output rejected; row → `failed` with reason

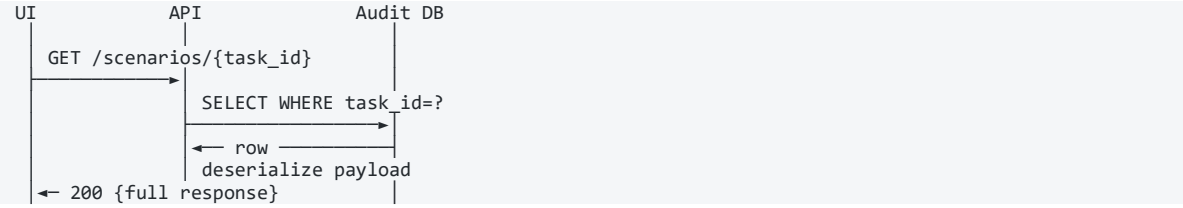
6.3 UC-03 — Generate batch of threat scenarios (sync)

Primary Actor: End User **Trigger:** User submits 1–50 fully-populated items to `POST /generate-scenario/batch`.



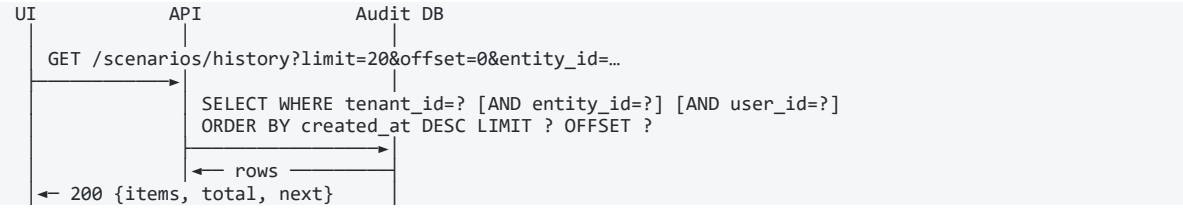
Note: No per-tenant concurrent limit is applied for batch; sliding-window read limiter does not apply (batch is a write endpoint).

6.4 UC-04 — Retrieve generated scenario by Task ID



Failure modes: 404 if `task_id` unknown; 200 with intermediate status if not yet `complete`.

6.5 UC-05 — Browse scenario history



6.6 UC-06 — Probe service health

Endpoint	Behaviour
`GET /health`	Returns 200 if the process is running; no dependency checks
`GET /health/ready`	Returns 200 only if DB, Redis broker, Redis result backend, cache backend, and LLM configuration are all healthy; 503 otherwise

6.7 Pipeline Engine Specifications

PHASE A · Intake	
1	**Input Capture** Analyst submits the 8 required inputs (asset, service, threat & control fields).
2	**Normalise & Validate** Canonicalise inputs; confirm every value exists in the knowledge base. *↳ branch: validation fails → HTTP 422, pipeline stops.*
PHASE B · Deterministic Reasoning (NO AI)	
3	**Build Context** Derive sector, asset criticality, business service and technology flags.
4	**Identify Threat** Match the input to a known threat record (exact / partial / semantic).
5	**Check Applicability** Seven-point relevance test plus technology gate. ***↳ branch: not applicable → HTTP 200 "Rejected" · LLM NOT called · pipeline stops.***
6	**Rank Risk**

	Compute likelihood & impact; assign priority — Critical / High / Medium / Low.
7	**Identify Controls** Map the controls expected to defend against the threat.
8	**Build Evidence & Gaps** List the audit evidence each identified control requires.
PHASE C · AI Narration (Controlled)	
9	**Build LLM Input Pack** Assemble approved facts only into a fixed, structured pack.
10	**Generate Scenario (LLM)** One governed AI call writes the readable defensive narrative.
11	**Validate Scenario** Post-AI validator cross-checks every name & term; scans for unsafe wording. *↳ branch: discrepancy found → flag "Human Review Required" (scenario still saved).*
PHASE D · Output & Record	
12	**Assemble Output & Record** Build attack tree, store full scenario + facts, return the result.
▼ Completed threat scenario returned to the analyst	

Figure 6-1 — Threat Scenario Logical Flow. The 12-step pipeline in four phases; Phases A, B and D are deterministic, the AI is invoked once in Phase C. Two early exits prevent wasted LLM calls. Native Word table — fully editable.

#	Engine	Input	Output	Side Effects
0	DB Enrichment	6 user fields	Enriched request with Sector, Sub-sector, Service, asset profile, controls, evidence, rules	Reads master-data cache
1	Normalizer	Enriched request	`NormalizedInput` (canonical strings, split CSVs, singularised actors)	None
2	Validator	`NormalizedInput`	`ValidationOutcome` (Passed / PassedWithWarnings / Failed)	Reads master DB
3	Context Builder	`NormalizedInput`	`ContextProfile` (asset + service + sector profile, `context_confidence`)	Reads master DB
4	Threat Identifier	`NormalizedInput` + `ContextProfile`	`IdentifiedThreat` (match status, confidence, security objective)	Reads master DB + embeddings
5	Applicability Engine	All prior	`ApplicabilityResult` (`Pass` / `Conditional` / `Rejected`, 7-check breakdown, score)	None
6	Scenario ID Builder	All prior	`generated_scenario_id` string	None
7	Ranking Engine	All prior	`RiskAssessment` (likelihood, impact, priority score, rating)	None
8	Control Identifier	All prior	`ControlProfile` (mapped controls, coverage score)	Reads master DB + embeddings
9	Evidence Builder	`ControlProfile`	`EvidenceRequirements` (aggregated artefacts)	None

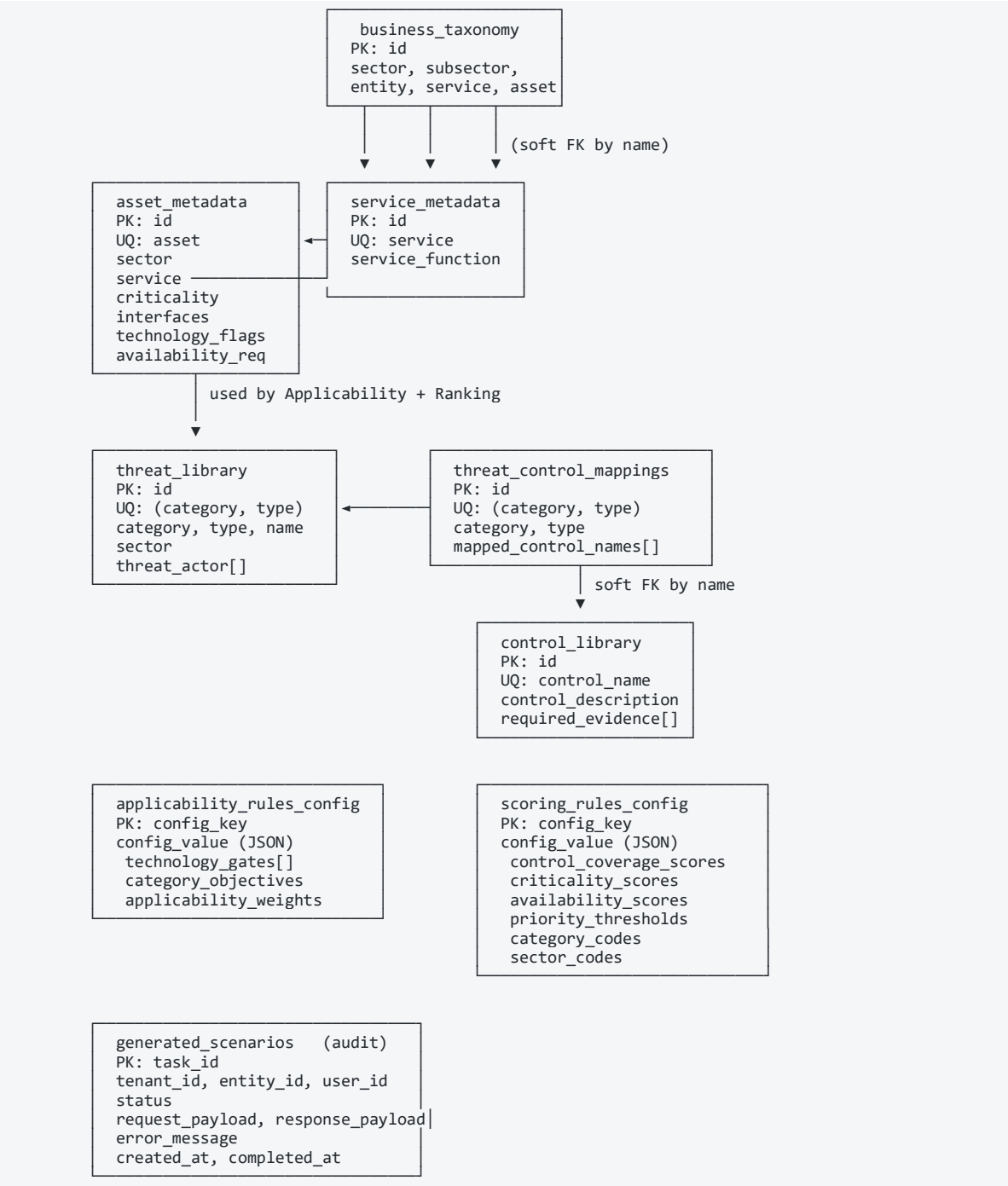
#	Engine	Input	Output	Side Effects
10	LLM Input Pack Builder	All prior	LLM input pack (dict with `generation_constraints`)	None
11	LLM Generator	LLM input pack	`ScenarioOutput`	Calls LLM provider (1 call)
—	Safety Validator	`ScenarioOutput` + all prior	`ScenarioValidationResult` (status, `human_review_required`)	None
12	Attack Tree Builder	`ScenarioOutput` + prior	`AttackTreeOutput` (nodes, edges, lanes)	None

Early-exit semantics:

Condition	Behaviour
Step 2 fails	`ScenarioValidationError` raised; HTTP 422
Step 5 returns `Rejected`	Pipeline exits after Step 6; LLM not called; response carries `status="Rejected"`, `scenario=None`, `attack_tree=None`; HTTP 200
Step 5 returns `Conditional`	Pipeline continues exactly as `Pass`; safety validator later sets `human_review_required=True`
Step 11 LLM fails	Circuit breaker opens after threshold; HTTP 502/503
Safety validator rejects	Response carries `status="Failed"`; HTTP 200 with failure reason

7. Data Design

7.1 Conceptual Data Model (Entity-Relationship Diagram)



7.2 Logical Data Model — Master Data Inventory

Entity	Purpose	Key	Cardinality
asset_metadata	Asset operational profile	asset (UQ)	One row per asset

Entity	Purpose	Key	Cardinality
service_metadata	Service profile	service (UQ)	One row per service
business_taxonomy	Hierarchical sector/subsector/entity/service/asset	id	Many rows per asset allowed
threat_library	Master threat catalogue	(category, type) (UQ)	One row per threat type
control_library	Master control catalogue	control_name (UQ)	One row per control
threat_control_mappings	Many-to-many threat → controls	(category, type) (UQ)	One row per threat type
applicability_rules_config	Technology gates + objectives + weights	config_key (PK)	One row per config key
scoring_rules_config	Numeric scoring lookups	config_key (PK)	One row per config key
generated_scenarios	Audit & runtime store	task_id (PK)	One row per submission

7.3 Physical Data Model

Concept	MSSQL	PostgreSQL	SQLite
Auto-increment PK	INT IDENTITY(1,1)	SERIAL	INTEGER AUTOINCREMENT
Variable-length string	NVARCHAR(n)	VARCHAR(n)	TEXT
Unbounded text / JSON	NVARCHAR(MAX)	TEXT	TEXT
Timestamp	DATETIME2(0)	TIMESTAMP	TIMESTAMP
UTC default	GETUTCDATE()	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP

Full DDL per dialect is provided in R4 (docs/SEED_DATA_DATABASE_SCHEMA.md).

7.4 Audit Data Model

```
CREATE TABLE generated_scenarios (
    task_id          NVARCHAR(36)    NOT NULL PRIMARY KEY,
    tenant_id        NVARCHAR(200)   NOT NULL,
    entity_id        NVARCHAR(200)   NULL,
    user_id          NVARCHAR(200)   NULL,
    status           NVARCHAR(20)    NOT NULL DEFAULT 'complete',
    request_payload  NVARCHAR(MAX)   NOT NULL,
    response_payload NVARCHAR(MAX)   NULL,
    error_message    NVARCHAR(MAX)   NULL,
    created_at       DATETIME2(0)    NOT NULL DEFAULT GETUTCDATE(),
    completed_at     DATETIME2(0)    NULL
);
CREATE INDEX idx_scenarios_tenant        ON generated_scenarios (tenant_id);
CREATE INDEX idx_scenarios_tenant_entity ON generated_scenarios (tenant_id, entity_id);
CREATE INDEX idx_scenarios_tenant_user   ON generated_scenarios (tenant_id, user_id);
CREATE INDEX idx_scenarios_status        ON generated_scenarios (status);
```

7.5 Output Data Model — Scenario

```
ScenarioOutput
├── scenario_title             : string
```

```
├── scenario_statement          : string
├── threat_actor                : list[string]
├── threat_category             : string
├── threat_type                 : string
├── threat_name                 : string
├── target_sector               : string
├── target_asset                : string
├── supported_service           : string
├── service_function            : string
├── high_level_attack_path      : string
├── attack_vectors              : list[string]      (2–6 items, feeds Vector nodes)
├── control_gaps                : list[string]      (2–6 items, feeds Condition nodes)
├── business_impact             : list[string]
├── operational_impact          : list[string]
├── cii_impact                  : list[string]
├── mapped_controls             : list[Control]
│   └── Control
│       ├── control_name        : string
│       ├── control_purpose     : string
│       └── required_evidence   : list[string]
├── required_evidence           : list[string]      (deduplicated aggregate)
├── control_gap_summary         : string
├── risk_summary
│   ├── priority_rating         : enum[Low, Medium, High, Critical]
│   ├── likelihood_score        : int (1–5)
│   ├── impact_score            : int (1–5)
│   └── residual_concern        : string
├── assumptions                 : list[string]
└── excluded_details            : list[string]      (e.g. payloads, exploit steps)
```

7.6 Output Data Model — Attack Tree

```
AttackTreeOutput
├── scenario_id                 : string  (= generated_scenario_id)
├── name                        : string
├── severity                    : enum[critical, high, medium, low]
├── status                      : enum[Active, Escalated, Monitoring]
├── affected                    : string
├── overview                    : string
├── critical_paths              : int
├── high_risk_conditions        : int
├── severity_meta               : string
├── tree_heading                : string
├── critical_node_id            : string
├── critical_node_ids           : list[string]
├── nodes                       : list[AttackTreeNode]
└── edges                       : list[AttackTreeEdge]

AttackTreeNode
├── id, type (actor|action|vector|condition|impact)
├── severity, posture
├── label, description, narrative
├── actor, action, vector, condition, impact (context strings)
├── actions (recommended)       : list[string]
├── path                        : list[string]
└── x, y, width, height         : int (canvas layout)

AttackTreeEdge
├── from_node                   : string
├── to_node                     : string
└── critical                    : bool
```

Lane mapping:

Lane	Node Type	Sourced From
1	Actor	`scenario.threat_actor[]`
2	Action	`scenario.threat_name`
3	Path / Vector	`scenario.attack_vectors[]`
4	Condition	`scenario.control_gaps[]`

Lane	Node Type	Sourced From
5	Impact	`scenario.business_impact + operational_impact + cii_impact`

7.7 Scenario Identifier Format

```
GEN-{SECTOR_CODE}-{ASSET_ID}-{CATEGORY_CODE}-{HASH6}
Example: GEN-IND-MES-DOS-4A2F1C
```

HASH6 = first six hex characters of SHA-256 over the JSON-serialised request (sorted keys). Stability is guaranteed: identical inputs produce identical IDs.

7.8 Risk Scoring Model

Component	Range	Formula
Likelihood	1–5	`mean(actor_capability, exposure_level, applicability_component)`
Impact	1–5	`max(criticality_score, service_score, availability_score)`
Priority Score	1–25	`Likelihood × Impact`
Priority Rating	enum	Low ≤5 · Medium 6–12 · High 13–19 · Critical ≥20

All component scores are derived from `scoring_rules_config`.

7.9 Data Flow Diagram

The data flow diagram below traces every data movement in the system. External entities are labelled E1–E3, processes P1–P4 and data stores D1–D4; the table lists each data flow in execution order.

External Entities	Processes	Data Stores
E1 Master Data Steward offline curator **E2** End User Web Browser **E3** LLM Provider OpenAI / Azure / Mock	**P1** Cascading Metadata API **P2** Scenario Submit API **P3** Pipeline Worker Steps 0–12 **P4** Poll API	**D1** Master DB **D2** Master-Data Cache **D3** Task Queue (Redis) **D4** Audit DB

#	From	Data Flow	To
Stage A — Master-data preparation (offline / worker startup)			
1	E1 Master Data Steward	→ *seed data + curated updates*	D1 Master DB
2	D1 Master DB	→ *master data — load-on-startup*	D2 Master-Data Cache
Stage B — Cascading metadata (picker reads)			
3	E2 End User (Browser)	→ *picker selection*	P1 Cascading Metadata API
4	D2 Master-Data Cache	→ *cached master data*	P1 Cascading Metadata API
5	P1 Cascading Metadata API	→ *filtered picker lists*	E2 End User (Browser)
Stage C — Scenario submission			

#	From	Data Flow	To
6	E2 End User (Browser)	→ *six selected fields*	P2 Scenario Submit API
7	P2 Scenario Submit API	→ *audit row (status = queued)*	D4 Audit DB
8	P2 Scenario Submit API	→ *task message*	D3 Task Queue (Redis)
9	P2 Scenario Submit API	→ *{ task_id, HTTP 202 Accepted }*	E2 End User (Browser)
Stage D — Asynchronous scenario generation			
10	D3 Task Queue (Redis)	→ *task message*	P3 Pipeline Worker
11	D2 Master-Data Cache	→ *master data — Step 0 enrichment*	P3 Pipeline Worker
12	P3 Pipeline Worker	→ *LLM input pack*	E3 LLM Provider
13	E3 LLM Provider	→ *structured scenario output*	P3 Pipeline Worker
14	P3 Pipeline Worker	→ *completed response payload*	D4 Audit DB
Stage E — Result retrieval			
15	E2 End User (Browser)	→ *GET /scenarios/{task_id}*	P4 Poll API
16	D4 Audit DB	→ *stored scenario payload*	P4 Poll API
17	P4 Poll API	→ *full scenario payload*	E2 End User (Browser)

Figure 7-1 — Data Flow Diagram. Every data movement between external entities (E1–E3), processes (P1–P4) and data stores (D1–D4), in execution order. Native Word table — fully editable.

7.10 Data Classification and Retention

Data	Classification	Retention	Where Stored
Master data (assets, threats, controls, rules)	Internal Confidential	Retained for service lifetime; versioned in source-controlled seeds	D1 + D2 cache
Submission requests	Tenant Confidential	Retained per client policy (default 24 months)	D4
Submission responses (scenarios, attack trees)	Tenant Confidential	Retained per client policy (default 24 months)	D4
LLM input pack (in transit)	Tenant Confidential	Not stored; ephemeral to LLM call	—
Logs	Operational	Per client log-management policy	Stdout → client log aggregator

8. Interface Design

8.1 Interface Inventory

ID	Direction	Channel	Format	Purpose
I-01	Inbound	HTTPS	JSON	Cascading metadata reads (UI ↔ API)
I-02	Inbound	HTTPS	JSON	Scenario submission and polling
I-03	Inbound	HTTPS	JSON	Health probes
I-04	Outbound	HTTPS	JSON	LLM provider call
I-05	Inbound (trust)	HTTPS header	Plain	`tenant_id` injected by client identity gateway
I-06	Outbound (optional)	HTTPS	JSON	Sentry / APM telemetry

8.2 API Interface Specifications

Common conventions:

- **Auth:** `tenant_id` injected as a trusted HTTP header by the client edge gateway.
- **Correlation:** Optional inbound `X-Request-ID` or W3C `traceparent`; one is generated if absent and echoed in the response.
- **Content type:** `application/json` request and response.
- **Errors:** Standard `{ "detail": "..." }` for 4xx/5xx.

8.2.1 `GET /metadata/assets`

Field	Value
Auth	tenant + optional `X-API-Key`
Query	none
Response 200	`{ "assets": ["Manufacturing Execution Platform", "Distribution SCADA", …] }`
Errors	401 missing key, 429 rate-limited

8.2.2 `GET /metadata/asset-types?asset={name}`

Field	Value
Query	`asset` (required)
Response 200	`{ "asset": "Manufacturing Execution Platform", "asset_types": ["OT/IT hybrid", "Cloud-native"] }`
Errors	404 asset unknown, 429

8.2.3 `GET /metadata/threat-categories?asset={name}&type={type}`

Field	Value
Query	`asset`, `type` (both required)
Response 200	`{ "categories": ["Denial of Service", "Tampering", "Insider Threat"] }`

8.2.4 GET /metadata/threat-types?category={cat}&sector={sec}

Field	Value
Query	`category`, `sector`
Response 200	`{ "threat_types": ["DDoS Attack on Nodes", "Volumetric Flood"] }`

8.2.5 GET /metadata/threats?type={type}

Field	Value
Query	`type`
Response 200	`{ "threats": ["Disruption of MES service nodes affecting production scheduling", …] }`

8.2.6 GET /metadata/threat-actors?threat={threat}

Field	Value
Query	`threat`
Response 200	`{ "threat_actors": ["Hacker", "Malicious insider"] }`

8.2.7 GET /metadata/asset-threats?asset={name} (Bulk-Mode helper)

Field	Value
Response 200	See Appendix A.1 for full sample

8.2.8 POST /generate-scenario

Request (six fields):

```
{
  "asset":           "Manufacturing Execution Platform",
  "asset_type":      "OT/IT hybrid",
  "threat_category": "Denial of Service",
  "threat_type":     "DDoS Attack on Nodes",
  "threat":          "Disruption of MES service nodes affecting production scheduling",
  "threat_actor":    ["Hacker", "Malicious insider"]
}
```

Response 202 Accepted:

```
{ "task_id": "3f2a4c1b-7e9d-4a2f-9c8e-1b6d3a5f7c2e" }
```

Errors: 413 size, 422 validation, 429 rate-limited, 503 LLM unconfigured.

8.2.9 GET /scenarios/{task_id}

Response 200 (status=complete): Full payload — see Appendix A.2. **Response 200 (status=queued/processing):** `{ "task_id": "...", "status": "processing" }` **Errors:** 404 unknown task.

8.2.10 POST /generate-scenario/batch

Request:

```
{ "items": [ { /* 6 fields */ }, ... up to 50 ] }
```

Response 200: `{ "results": [ /* one response per item */ ] }`

8.2.11 `GET /scenarios/history`

Field	Value
Query	`limit` (default 20, max 100), `offset`, optional `entity_id`, `user_id`, `status`, `from`, `to`
Response 200	`{ "items": [...], "total": N, "next_offset": M }`

8.2.12 `GET /health` — Liveness

Response 200: `{ "status": "ok" }`

8.2.13 `GET /health/ready` — Readiness

Response 200: `{ "status": "ok", "checks": { "db": "ok", "broker": "ok", "result": "ok", "cache": "ok", "llm": "configured" } }` **Response 503:** `{ "status": "degraded", "checks": { "broker": "down" } }`

8.3 LLM Provider Interface (I-04)

Aspect	Specification
Direction	Outbound from worker to LLM provider
Protocol	HTTPS (TLS 1.2+)
Authentication	Provider API key (env-injected)
Request	Constrained input pack (no raw user input, no master data beyond approved enrichment)
Response	Structured `ScenarioOutput` (validated against Pydantic schema)
Timeout	`LLM_REQUEST_TIMEOUT_SECONDS` (default 60 s)
Failure handling	Circuit breaker (closed → open after `LLM_CIRCUIT_FAILURE_THRESHOLD` consecutive failures; half-open after `LLM_CIRCUIT_OPEN_SECONDS`)
Concurrency	One call per pipeline execution; no batching

8.4 Identity Gateway Interface (I-05)

Aspect	Specification
Direction	Inbound (client edge → API)
Trust model	The API trusts the gateway-injected `tenant_id` header; gateway must not be bypassable
Required headers	`X-Tenant-ID` (mandatory); `X-User-ID`, `X-Entity-ID` (optional)
Client responsibility	Verify the user's token, derive `tenant_id`, reject mismatched submissions (see §13 D1)

8.5 Cross-Cutting Interface Concerns

Concern	Specification
Request size	`MAX_REQUEST_BYTES` (default 256 KB); HTTP 413 on excess

Concern	Specification
CORS	`CORS_ALLOWED_ORIGINS` env-driven; wildcard rejected in production
Rate limit (concurrent)	`TENANT_MAX_ACTIVE_TASKS` (default 20); HTTP 429
Rate limit (read window)	`READ_RATE_LIMIT_MAX_PER_WINDOW` per `READ_RATE_LIMIT_WINDOW_SECONDS`; HTTP 429
Metadata gate	`PUBLIC_API_KEY` (optional); when set, all `/metadata/*` require `X-API-Key`
Correlation	`X-Request-ID` and W3C `traceparent` honoured; UUID generated if absent

9. Security Architecture

9.1 Authentication and Authorisation

Concern	Design
End-user authentication	Delegated to client IdP via edge gateway (federation, SSO, mTLS)
Tenant identification	Gateway injects verified `tenant_id` on every request
Authorisation scope	All read and write operations are tenant-scoped via middleware
Service-to-service	Optional `X-API-Key` gate on read endpoints
LLM credentials	API key injected via environment from secrets manager

9.2 Data Protection

State	Control
In transit (UI ↔ API, API ↔ LLM)	TLS 1.2+ enforced
In transit (within deployment)	Operator responsibility (e.g. service mesh / mTLS)
At rest (Master DB)	Database-native encryption (operator-configured)
At rest (Audit DB)	Database-native encryption (operator-configured)
At rest (Redis)	Redis ACL + AT rest encryption (operator-configured)
Logs	No secrets; log scrubbing on known keys

9.3 Audit and Logging

Event	Persisted To	Detail
Submission received	`generated_scenarios` (status=queued)	`task_id`, `tenant_id`, `request_payload`, `created_at`
Submission processing	`generated_scenarios` (status=processing)	updated_at
Submission complete	`generated_scenarios` (status=complete)	`response_payload`, `completed_at`
Submission failed	`generated_scenarios` (status=failed)	`error_message`, `completed_at`
API request	Structured JSON log	request-id, method, path, status, latency, tenant_id
LLM call	Structured JSON log	request-id, duration, token usage (if reported by provider), success/failure

9.4 LLM Security Controls

Control	Mechanism
Constrained input	LLM receives only the input pack assembled by Step 10; no raw user input

Control	Mechanism
Generation constraints	Hard `generation_constraints` instruct the model not to invent, not to provide exploit steps
Output cross-check	Post-LLM validator confirms every asset, control, actor in the output exists in master data
Unsafe-term scanner	Rejects payloads, exploit steps, bypass techniques, credential-theft instructions (policy-driven)
Single call	One LLM invocation per scenario; no agentic loops
Circuit breaker	Fails fast on degraded provider

9.5 Network Security Zones

Zone	Components	Allowed Flows
Public	End User browser	→ Client Edge Gateway (HTTPS)
Edge	Client Identity Gateway, optional WAF	→ API zone (HTTPS, internal)
Application	API service, Worker pool	→ Data zone, Cache zone, External zone (egress only)
Data	Master DB, Audit DB	Inbound from Application only
Cache	Redis	Inbound from Application only
External	LLM provider	Egress only from Application

9.6 Threat Model Summary

Threat	Mitigation
Hallucinated asset/control in output	Output cross-check validator + master-data constraint on inputs
Prompt injection via user input	No free-text input; pickers only; canonical normalisation; no user text reaches the LLM verbatim
Cross-tenant data exposure	Tenant-scoped middleware on all endpoints; per-tenant audit isolation
LLM provider compromise	Hardened input pack; output validator; secrets rotation (client D2)
Denial of service	Concurrent task limit + sliding-window read limiter + request size guard
Repudiation of scenario authorship	Full audit row with `user_id`, `tenant_id`, `request_payload`, `created_at`
Replay attack on async polling	Optional `X-API-Key`; tenant-scoping (subject to client D1)

10. Non-Functional Design

10.1 Performance Design

Path	Design Choice	Target
Metadata read	In-memory cache lookup	P95 ≤ 200 ms
Scenario generation	Single LLM call after deterministic enrichment	P95 ≤ 30 s
Polling	Indexed read from audit DB	P95 ≤ 100 ms
Cold start (worker)	Master-data cache + embedding model load	≤ 60 s

10.2 Scalability Design

Dimension	Approach
API horizontal scale	Stateless service behind load balancer
Worker horizontal scale	Add Celery worker processes; broker is the coordination point
Cache scaling	Switch from `InProcessCache` to `RedisCache` for cross-process sharing
Tenant fairness	Per-tenant concurrent task limit + per-(tenant, route) read window
Database	Connection pool tuning via `DB_POOL_SIZE`, `DB_MAX_OVERFLOW`, `DB_POOL_RECYCLE_SECONDS`, `DB_POOL_TIMEOUT_SECONDS`

10.3 Availability and Disaster Recovery

Aspect	Design
Liveness	`GET /health`
Readiness	`GET /health/ready` (DB + broker + result + cache + LLM config)
Single-node failure	Stateless API replaceable; workers can be killed mid-task without data loss (task re-enqueues on broker visibility timeout)
LLM provider outage	Circuit breaker isolates; runbook `docs/RUNBOOKS/llm_outage.md`
Redis outage	Tasks block; runbook `docs/RUNBOOKS/redis_outage.md`
DB failover	Connection pool recycles; runbook `docs/RUNBOOKS/db_failover.md`
Worker stuck	Visibility timeout; runbook `docs/RUNBOOKS/worker_stuck.md`
Backup / DR	Operator responsibility per `docs/RUNBOOKS/backup_dr.md`; RPO/RTO defined by client backup policy

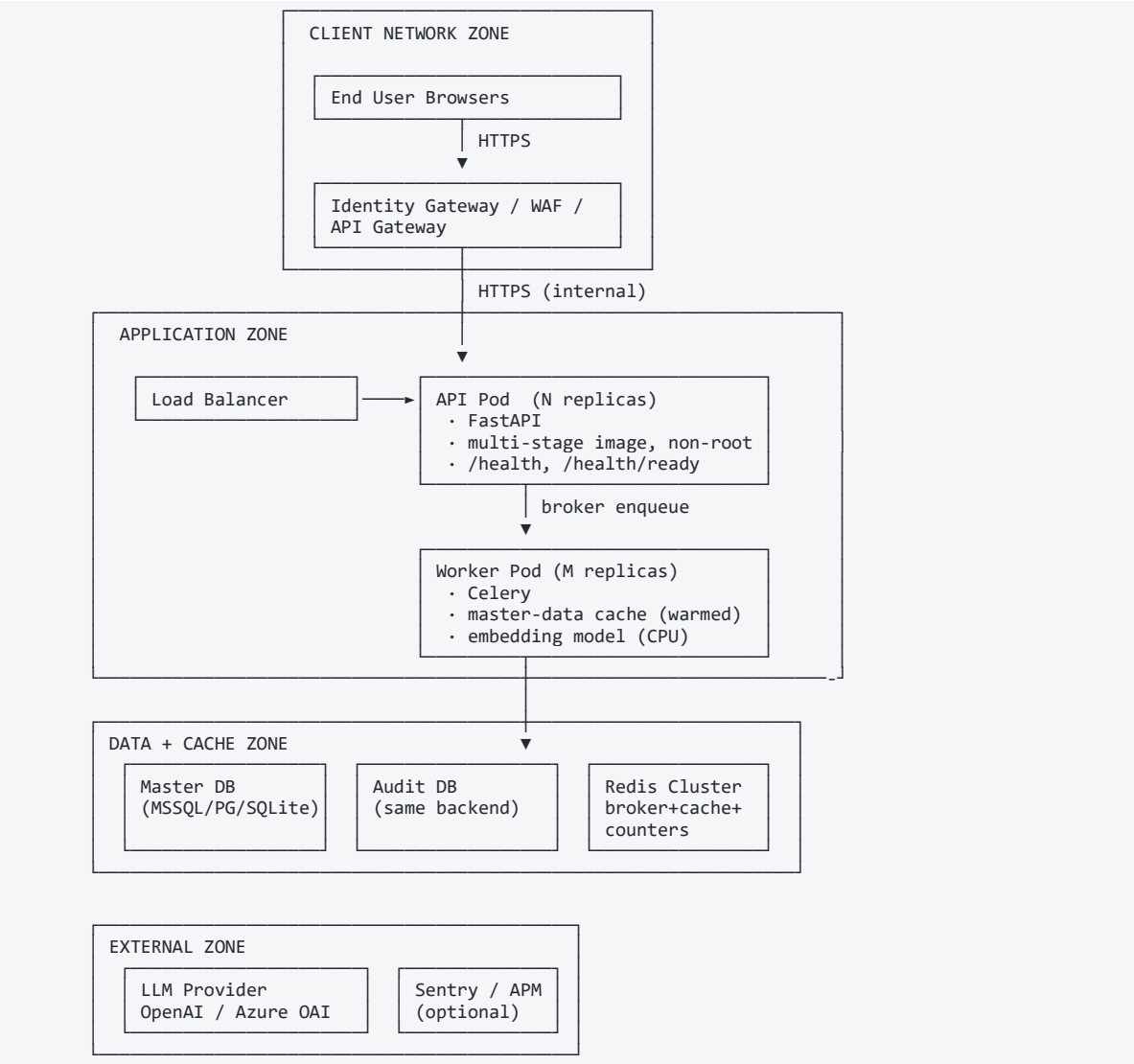
10.4 Observability Design

Capability	Mechanism
Structured logs	JSON formatter in production (`LOG_FORMAT=json`)
Request correlation	`X-Request-ID` + W3C `traceparent`, propagated through Celery

Capability	Mechanism
Metrics	Exported via standard middleware; collected by client observability stack
Tracing	Optional Sentry SDK (FastAPI / Celery / SQLAlchemy / Redis integrations)
Health	`/health`, `/health/ready`
Audit KPIs	Queryable from `generated_scenarios` (volume, latency, rejection rate, failure rate)

11. Deployment Architecture

11.1 Deployment Topology Diagram



11.2 Environment Configuration

Variable	Purpose	Default
`LLM_PROVIDER`	LLM selector	`mock`
`OPENAI_API_KEY` / `AZURE_OPENAI_*`	Provider credentials	—
`LLM_REQUEST_TIMEOUT_SECONDS`	Per-call timeout	60
`LLM_CIRCUIT_FAILURE_THRESHOLD`	Breaker open threshold	5
`LLM_CIRCUIT_OPEN_SECONDS`	Breaker open duration	30
`RDBMS_BACKEND` / `DATABASE_URL`	Database selector	`sqlite`
`DB_POOL_SIZE`	Connection pool size	10
`DB_MAX_OVERFLOW`	Pool overflow	20

Variable	Purpose	Default
DB_POOL_RECYCLE_SECONDS	Connection recycle	1800
DB_POOL_TIMEOUT_SECONDS	Connection wait timeout	30
REDIS_URL	Broker + cache	redis://localhost:6379/0
CACHE_BACKEND	memory or redis	memory
CACHE_TTL_SECONDS	Cache TTL	3600
CACHE_SCHEMA_VERSION	Cache namespace	v1
TENANT_MAX_ACTIVE_TASKS	Concurrent task cap	20
READ_RATE_LIMIT_MAX_PER_WINDOW	Sliding window burst	120
READ_RATE_LIMIT_WINDOW_SECONDS	Sliding window size	60
MAX_REQUEST_BYTES	Request size cap	262144
CORS_ALLOWED_ORIGINS	CORS allow-list	—
PUBLIC_API_KEY	Metadata gate	—
LOG_FORMAT	auto / json / text	auto
SENTRY_DSN	APM endpoint	—
APP_ENV	Environment marker	dev

11.3 Container Specification

Aspect	Value
Base image	python:3.12-slim
Build	Multi-stage (build + runtime)
Runtime user	app (UID 10001), non-root
Healthcheck	GET /health
Exposed port	8000
Entrypoint (API)	uvicorn app.main:app --host 0.0.0.0 --port 8000
Entrypoint (Worker)	celery -A app.infrastructure.queue.celery_app worker

12. Operations and Monitoring

Operational Aspect	Reference
Liveness probe	`GET /health`
Readiness probe	`GET /health/ready`
Log format	JSON in production
Request correlation	`X-Request-ID`, W3C `traceparent`
Metrics surface	Custom middleware metrics; pulled by client observability stack
Trace export	Sentry SDK (opt-in)
Runbooks	`docs/RUNBOOKS/llm_outage.md`, `redis_outage.md`, `worker_stuck.md`, `db_failover.md`, `backup_dr.md`
Load test	`scripts/loadtest/` (Locust)
Dependency audit	`scripts/audit_deps.py` (wraps `pip-audit`)

13. Risks and Mitigations

ID	Risk	Likelihood	Impact	Mitigation
RSK-01	LLM provider outage delays scenario generation	Medium	Medium	Circuit breaker; queued tasks retry; runbook documented
RSK-02	Master data becomes stale relative to live environment	Medium	High	Operational data-steward process; cache schema versioning forces refresh
RSK-03	Tenant-ID spoofing if edge gateway is bypassed	Low	High	Client D1 — gateway must verify and inject `tenant_id`; documented in §13.D1
RSK-04	LLM cost overrun under load	Medium	Medium	Per-tenant concurrent limit; sliding-window read limit; circuit breaker
RSK-05	Embedding model cold-start increases first-request latency	Medium	Low	Workers pre-warm cache and model; readiness probe gates traffic until ready
RSK-06	Single-region deployment vulnerable to regional outage	Variable	High	Out of scope; client's deployment topology decision
RSK-07	Audit DB growth outpaces storage budget	Medium	Low	Client retention policy defines archival; partitioning recommended >12 mo
RSK-08	Master data update bypasses cache invalidation	Low	Medium	Cache namespace tied to `CACHE_SCHEMA_VERSION`; bump on data update

14. Assumptions, Dependencies, and Constraints

14.1 Assumptions

ID	Statement
A-01	The client identity gateway authenticates end users and injects verified `tenant_id`
A-02	Master data is curated and maintained by the client's security data team
A-03	A reachable LLM provider exists, or the Mock provider is acceptable
A-04	The UI enforces selection-only input; direct API integrators send DB-sourced values
A-05	A single LLM call per scenario is acceptable in the cost/latency budget
A-06	Secrets (LLM key, DB password) are injected at deploy time from the client's secrets manager

14.2 Dependencies

ID	Dependency
DEP-01	Client identity provider (SSO / mTLS)
DEP-02	LLM provider (OpenAI / Azure OpenAI) — or Mock
DEP-03	Database (MSSQL / PostgreSQL / SQLite)
DEP-04	Redis (broker, cache, counters)
DEP-05	Container orchestrator (Kubernetes or equivalent)

14.3 Constraints

ID	Constraint
CON-01	Batch endpoint accepts at most 50 items per request
CON-02	Maximum request body 256 KB (configurable)
CON-03	Per-tenant concurrent task limit 20 (configurable)
CON-04	Read endpoints capped at 120 requests / 60 s per (tenant, route) (configurable)
CON-05	Master data is read-only at runtime; updates require seed reload or worker restart

14.4 Client Responsibilities (Deferred Items)

ID	Deferred Item	Recommended Client Action
D1	Tenant ownership enforcement on `GET /scenarios/{task_id}` and `/scenarios/history`	Front the API with an authenticated edge that verifies `tenant_id` from the token
D2	Secret rotation for LLM key and database password	Inject from secrets manager at deploy; rotate per client cadence
D3	CI/CD pipeline	Wrap shipped `pytest` suite and dependency-audit script in client's CI
D4	Observability backend connection	Set `SENTRY_DSN` and route structured logs to client log aggregator

15. Open Issues

ID	Issue	Owner	Target Resolution
OPEN-01	Asset Type taxonomy — confirm whether to extend `asset_metadata` with a dedicated `asset_type` column or to derive from `environment_type`	Client Data Steward	Before UAT
OPEN-02	Read-endpoint API-key issuance and rotation policy	Client Security	Before production cutover
OPEN-03	Audit retention period and archival mechanism	Client Compliance	Before production cutover
OPEN-04	LLM provider selection (OpenAI vs Azure OpenAI) and region	Client Procurement	Before production cutover
OPEN-05	Monitoring and alerting endpoints (Sentry DSN, log aggregator URL)	Client Infrastructure	Before production cutover
OPEN-06	Master-data update cadence and SLA	Client Data Steward	Before UAT

16. Appendices

Appendix A — Sample Payloads

A.1 Sample `GET /metadata/asset-threats?asset=Manufacturing+Execution+Platform`


```
{
  "asset": "Manufacturing Execution Platform",
  "service": "Manufacturing Execution and Production Scheduling",
  "sector": "Industry",
  "threat_count": 3,
  "threats": [
    {
      "threat_category": "Denial of Service",
      "threat_type": "DDoS Attack on Nodes",
      "threat_name": "Disruption of MES service nodes affecting production scheduling",
      "threat_actor": ["Hacker", "Malicious insider"],
      "control_name": ["High Availability Architecture", "Network Segmentation", "Tested Failover"],
      "excluded_reason": null
    },
    {
      "threat_category": "Tampering",
      "threat_type": "Configuration Tampering",
      "threat_name": "Unauthorised change to MES production configuration",
      "threat_actor": ["Malicious insider"],
      "control_name": ["Change Management Controls", "Configuration Baseline", "Privileged Access
Management"],
      "excluded_reason": null
    }
  ],
  "excluded_threats": 1,
  "excluded": [
    {
      "threat_type": "51% Attack",
      "excluded_reason": "Technology gate: uses_blockchain is not true for this asset"
    }
  ]
}
```


A.2 Sample `GET /scenarios/{task_id}` (status=complete)


```
{
  "task_id": "3f2a4c1b-7e9d-4a2f-9c8e-1b6d3a5f7c2e",
  "status": "Generated",
  "generated_scenario_id": "GEN-IND-MES-DOS-4A2F1C",
  "scenario": {
    "scenario_title": "DDoS-Driven Disruption of Manufacturing Execution Platform",
    "scenario_statement": "A volumetric DDoS attack targeting MES service nodes could disrupt real-time
production scheduling…",
    "threat_actor": ["Hacker", "Malicious insider"],
    "threat_category": "Denial of Service",
    "threat_type": "DDoS Attack on Nodes",
    "threat_name": "Disruption of MES service nodes affecting production scheduling",
    "target_sector": "Industry",
    "target_asset": "Manufacturing Execution Platform",
    "supported_service": "Manufacturing Execution and Production Scheduling",
    "high_level_attack_path": "Attacker floods SCADA-facing network interfaces with volumetric traffic…",
    "attack_vectors": [
      "Attacker floods SCADA-facing network interfaces with volumetric traffic",
      "MES nodes exhaust processing capacity responding to malformed packets",
      "Legitimate production control commands are queued and dropped"
    ],
    "control_gaps": [
      "Failover procedures exist but have not been tested under sustained volumetric load",
      "No traffic rate-limiting configured on SCADA network segments"
    ],
    "business_impact": ["Production schedule disruption", "Revenue loss per hour of downtime"],
    "operational_impact": ["Loss of real-time OEE visibility"],
    "cii_impact": ["Degradation of industrial production CII service"],
    "mapped_controls": [
      { "control_name": "High Availability Architecture",
        "control_purpose": "Ensures MES nodes can absorb or failover under load",
        "required_evidence": ["Architecture diagram", "Failover test results"] }
    ],
    "control_gap_summary": "Failover procedures exist but evidence of testing under sustained load is not
available.",
```

```
    "risk_summary": {
      "priority_rating": "Critical",
      "likelihood_score": 4,
      "impact_score": 5,
      "residual_concern": "Untested failover under sustained load"
    },
    "assumptions": ["Network interfaces are reachable from external segments"],
    "excluded_details": ["Specific payload characteristics", "Step-by-step attack execution"]
  },
  "attack_tree": {
    "scenario_id": "GEN-IND-MES-DOS-4A2F1C",
    "name": "DDoS-Driven Disruption of MES",
    "severity": "critical",
    "status": "Active",
    "nodes": [ "..." ],
    "edges": [ "..." ]
  },
  "risk_assessment": { "likelihood_score": 4, "impact_score": 5, "priority_score": 20, "priority_rating":
"Critical" },
  "validation_result": { "validation_status": "Passed", "human_review_required": true }
}
```


A.3 Sample Rejection Response


```
{
  "task_id": "9b1e3c2d-…",
  "status": "Rejected",
  "generated_scenario_id": "GEN-IND-MES-CRY-7D9E2A",
  "scenario": null,
  "attack_tree": null,
  "applicability_result": {
    "status": "Rejected",
    "rejection_reason": "Technology gate: the selected asset does not have uses_blockchain=true."
  }
}
```


Appendix B — Glossary

Term	Definition
CII	Critical Information Infrastructure
Master Data	Curated reference catalogue (assets, threats, controls, rules)
Pre-Step	The `GET /metadata/asset-threats` Bulk-Mode helper
Pipeline	Steps 0–12 plus Safety Validator
Applicability	Step 5 — the engine deciding whether a threat is relevant to an asset
Posture	Control coverage classification — Strong / Partial / Moderate / Low
Scenario ID	`GEN-{SECTOR}-{ASSET}-{CATEGORY}-{HASH}`
Task ID	UUID assigned to each asynchronous submission
Attack Tree	Five-lane node/edge graph (Actor → Action → Vector → Condition → Impact)
Critical Path	The single highlighted chain from Actor to Impact through the primary vector and primary control gap
Circuit Breaker	Process-local guard around the LLM call (closed / open / half-open)
Readiness Probe	`GET /health/ready` — deep dependency check

End of Solution Design Document.